

MECHATRONICS AND IOT

ASSIGNMENT - 7

SMART ELEVATOR AUTOMATION AND HUMAN DETECTION SYSTEM USING PIR SENSOR

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

SMART ELEVATOR AUTOMATION AND HUMAN DETECTION SYSTEM USING PIR SENSOR

1. Project Overview

The Smart Elevator Automation and Human Detection System is an IoT-based automation project designed to improve the safety, convenience, and efficiency of elevator operation. The system uses a PIR (Passive Infrared) sensor to detect the presence of a person near or inside the elevator area.

An **ESP8266 NodeMCU** acts as the main controller and processes the signal received from the PIR sensor. Based on human detection and user input through push buttons, the controller operates a **servo motor** to represent elevator door movement. LEDs indicate the elevator status, while a buzzer provides an audio indication during operation.

The proposed system can be simulated using electronic components and programmed through the Arduino IDE. The prototype demonstrates the basic working principle of a smart elevator without requiring a real elevator mechanism.

Main components:

- Human presence detection using PIR sensor.
- Automatic elevator door control.
- Floor selection using push buttons.
- LED-based status indication.
- Buzzer-based warning/arrival indication.
- Servo motor for elevator door simulation.
- ESP8266 NodeMCU as the main controller.
- Low-cost and easy-to-build prototype.
- Improved safety through human detection.

2. Project Statement

Elevators are widely used in residential buildings, offices, hospitals, shopping malls, educational institutions, and industrial facilities. Conventional elevator systems mainly depend on push buttons, door-control mechanisms, and basic safety sensors for their operation. Although these systems provide reliable transportation between floors, they may not adequately demonstrate intelligent human-presence detection and automated decision-making in low-cost elevator prototypes.

Another problem is that conventional small-scale elevator models generally require manual control. The user must operate switches or buttons to control the elevator, while the system does not automatically respond to the presence of a person.

There is also a need for a simple, economical, and easily programmable human-detection system for educational and prototype applications. Commercial elevator safety systems can involve sophisticated sensors and controllers, making them expensive and difficult to reproduce in a student-level project.

3. Proposed Solution

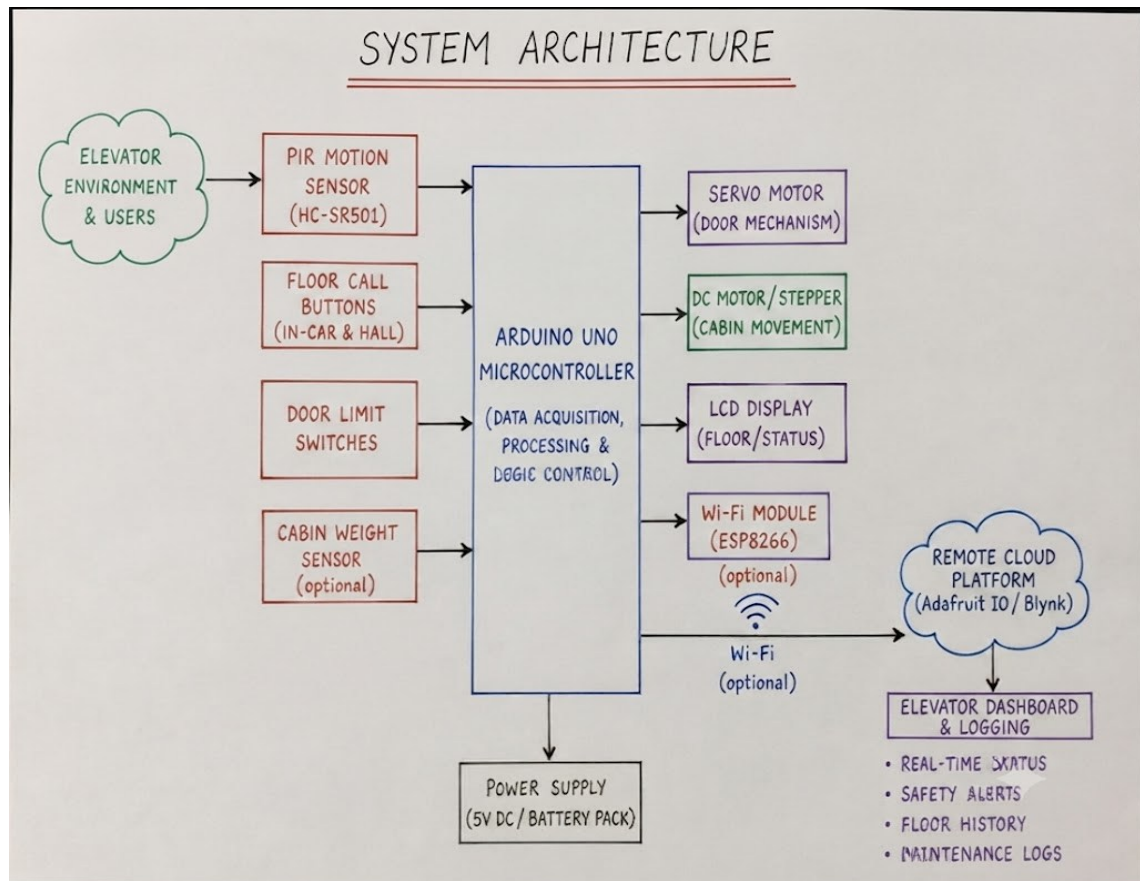
The proposed system uses a **PIR sensor** to detect human movement or presence. The PIR sensor continuously monitors the elevator entrance or cabin area.

The system operates as follows:

1. PIR sensor generates a HIGH signal.
2. ESP8266 receives the signal.
3. The controller processes the input.
4. The servo motor opens the elevator door.
5. Green LED indicates safe/active operation.
6. The user can select the required floor using push buttons.

7. After the required operation, the servo closes the door.
8. A buzzer provides an audio indication.

4. System Architecture:

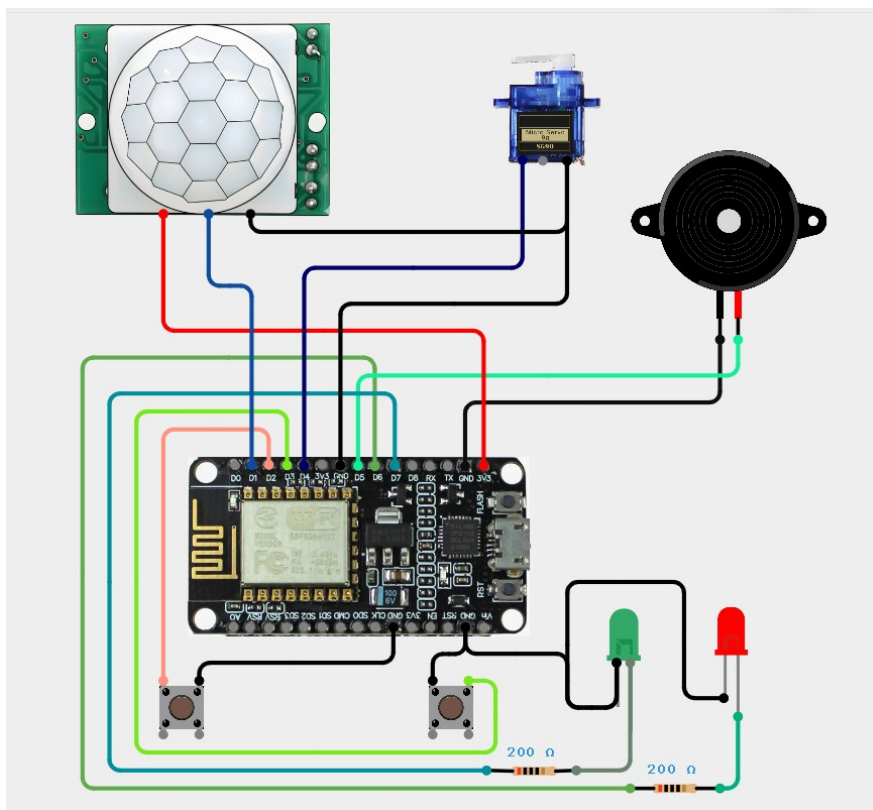


5. Circuit Table:

CIRCUIT TABLE

S. No.	Component	Pin	ESP8266 NodeMCU Pin	Description
1.	MQ135 Gas Sensor	VCC	VIN (5V)	Power Supply (5V)
		GND	GND	Ground
		A0	A0	Analog Output
2.	DHT11 Temperature & Humidity Sensor	VCC	3.3V	Power Supply (3.3V)
		GND	GND	Ground
		DATA	D2	Digital Data
3.	MPU6050 (Motion / Fall Detection Sensor)	VCC	3.3V	Power Supply (3.3V)
		GND	GND	Ground
		SCL	D1 (SCL)	I2C Clock
		SDA	D2 (SDA)	I2C Data
4.	Buzzer	+ (Positive)	D5	Buzzer Control
		- (Negative)	GND	Ground
5.	Green LED	Anode (+)	D6 through 220Ω	Green LED (Safe)
		Cathode (-)	GND	Ground
6.	Red LED	Anode (+)	D7 through 220Ω	Red LED (Unsafe)
		Cathode (-)	GND	Ground
7.	Power Supply	+5V	VIN (5V)	5V DC Supply
		GND	GND	Ground

6. Circuit Design:



7. Algorithm:

Step 1: Start the system.

Step 2: Initialize the PIR sensor, floor sensors, buttons, motor driver, display, and other components.

Step 3: Check whether a person is detected.

Step 4: If no person is detected, keep the elevator in standby mode.

Step 5: If a person is detected, activate the elevator control system.

Step 6: Read the floor-selection input.

Step 7: Compare the current floor with the selected floor.

Step 8: If the selected floor is above the current floor, rotate the motor in the upward direction.

Step 9: If the selected floor is below the current floor, rotate the motor in the downward direction.

Step 10: Continuously monitor the floor sensors.

Step 11: When the selected floor is detected, stop the motor.

Step 12: Activate the door-opening mechanism.

Step 13: Monitor human movement and door safety conditions.

Step 14: Close the door after the appropriate condition is satisfied.

Step 15: Return the elevator to standby mode.

Step 16: Repeat the process for the next request.

8. Coding:

```
#include "AdafruitIO_WiFi.h"
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>

#include <DHT.h>

#include <TinyGPSPlus.h>

#include <HardwareSerial.h>

// =====

// WIFI DETAILS

// =====

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"

#define WIFI_PASSWORD "qkts5737"

// =====

// ADAFRUIT IO DETAILS

// =====

#define IO_USERNAME  "Sivasaravanan"

#define IO_KEY       "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"

// Create Adafruit IO WiFi object

AdafruitIO_WiFi io(

  IO_USERNAME,

  IO_KEY,

  WIFI_SSID,

  WIFI_PASSWORD

);

// =====

// FEEDS

// =====

AdafruitIO_Feed *pulseFeed =

  io.feed("pulse");

AdafruitIO_Feed *temperatureFeed =

  io.feed("temperature");

AdafruitIO_Feed *humidityFeed =
```

```
    io.feed("humidity");
AdafruitIO_Feed *mq2Feed =
    io.feed("mq2");
AdafruitIO_Feed *latitudeFeed =
    io.feed("latitude");
AdafruitIO_Feed *longitudeFeed =
    io.feed("longitude");
AdafruitIO_Feed *satellitesFeed =
    io.feed("satellites");
AdafruitIO_Feed *gasAlertFeed =
    io.feed("gas-alert");

// =====
// DHT11
// =====

#define DHT_PIN 4
#define DHT_TYPE DHT11
DHT dht(DHT_PIN, DHT_TYPE);
// =====

// PULSE SENSOR
// =====

#define PULSE_PIN 34
// =====

// MQ2 GAS SENSOR
// =====

#define MQ2_PIN 35
// =====

// BUZZER
// =====

#define BUZZER_PIN 25
```



```
// =====  
// OLED  
// =====  
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64  
Adafruit_SSD1306 display(  
  SCREEN_WIDTH,  
  SCREEN_HEIGHT,  
  &Wire,  
  -1  
);  
// =====  
// GPS  
// =====  
#define GPS_RX 16  
#define GPS_TX 17  
HardwareSerial GPS(2);  
TinyGPSPlus gps;  
// =====  
// VARIABLES  
// =====  
float temperature = 0;  
float humidity = 0;  
int pulseValue = 0;  
int mq2Value = 0;  
float latitude = 0;  
float longitude = 0;  
int satellites = 0;  
bool gasAlert = false;
```

```

// =====
// TIMING
// =====

unsigned long lastSend = 0;

const unsigned long sendInterval =

    15000;

// =====

// SETUP
// =====

void setup()

{

    Serial.begin(115200);

    delay(1000);

    // -----

    // DHT

    // -----

    dht.begin();

    // -----

    // SENSOR PINS

    // -----

    pinMode(PULSE_PIN, INPUT);

    pinMode(MQ2_PIN, INPUT);

    pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(

        BUZZER_PIN,

        LOW

    );

    // -----

    // OLED

```

```
// -----  
Wire.begin(21, 22);  
if (!display.begin(  
    SSD1306_SWITCHCAPVCC,  
    0x3C))  
{  
    Serial.println(  
        "OLED not detected!"  
    );  
}  
else  
{  
    display.clearDisplay();  
    display.setTextColor(  
        SSD1306_WHITE  
    );  
    display.setTextSize(1);  
    display.setCursor(10, 10);  
    display.println(  
        "SMART MONITOR"  
    );  
    display.setCursor(10, 30);  
  
    display.println(  
        "Starting..."  
    );  
    display.display();  
}  
// -----
```

```
// GPS
// -----
GPS.begin(
  9600,
  SERIAL_8N1,
  GPS_RX,
  GPS_TX
);
// -----
// CONNECT TO ADAFRUIT IO
// -----
Serial.println();
Serial.println(
  "Connecting to Adafruit IO..."
);
io.connect();
// Wait for connection
while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}
Serial.println();
Serial.println(
  "Connected to Adafruit IO!"
);
display.clearDisplay();
display.setTextSize(1);
display.setCursor(0, 10);
```

```
display.println(
  "Adafruit IO"
);
display.setCursor(0, 30);
display.println(
  "Connected!"
);
display.display();
delay(2000);
}
// =====
// LOOP
// =====
void loop()
{
  // Keep Adafruit IO connection alive
  io.run();
  // -----
  // READ GPS
  // -----
  while (GPS.available())
  {
    gps.encode(
      GPS.read()
    );
  }
  if (gps.location.isValid())
  {
    latitude =
```

```

    gps.location.lat();
    longitude =
    gps.location.lng();
}
if (gps.satellites.isValid())
{
    satellites =
    gps.satellites.value();
}
// -----
// SEND SENSOR DATA
// -----
if (
    millis() - lastSend >=
    sendInterval
)
{
    lastSend = millis();
    // =====
    // DHT11
    // =====
    temperature =
    dht.readTemperature();
    humidity =
    dht.readHumidity();
    if (isnan(temperature) ||
        isnan(humidity))
    {
        Serial.println(

```

```
        "DHT11 reading failed!"
    );
}
else
{
    Serial.print(
        "Temperature: "
    );
    Serial.print(
        temperature
    );
    Serial.println(" C");
    Serial.print(
        "Humidity: "
    );
    Serial.print(
        humidity
    );
    Serial.println(" %");
}
// =====
// PULSE SENSOR
// =====
pulseValue =
    analogRead(PULSE_PIN);
Serial.print(
    "Pulse Sensor: "
);
Serial.println(
```

```
    pulseValue
  )
  // =====
  // MQ2
  // =====
  mq2Value =
    analogRead(MQ2_PIN);
  Serial.print(
    "MQ2: "
  );
  Serial.println(
    mq2Value
  )
  // =====
  // GAS ALERT
  // =====

  // Adjust this threshold after testing
  if (mq2Value > 2000)
  {
    gasAlert = true;
    digitalWrite(
      BUZZER_PIN,
      HIGH
    )
  }
  else
    gasAlert = false;
    digitalWrite(
```



```
BUZZER_PIN,  
LOW  
);  
}  
// =====  
// GPS SERIAL OUTPUT  
// =====  
Serial.print(  
  "Latitude: "  
);  
Serial.println(  
  latitude,  
  6  
);  
Serial.print(  
  "Longitude: "  
);  
Serial.println(  
  longitude,  
  6  
);  
Serial.print(  
  "Satellites: "  
)  
Serial.println(  
  satellites  
);  
// =====  
// SEND TO ADAFRUIT IO
```

```
// =====  
Serial.println();  
Serial.println(  
    "Sending data to Adafruit IO..."  
);  
if (!isnan(temperature))  
{  
    temperatureFeed->save(  
        temperature  
    );  
}  
if (!isnan(humidity))  
{  
    humidityFeed->save(  
        humidity  
    );  
}  
pulseFeed->save(  
    pulseValue  
);  
mq2Feed->save(  
    mq2Value  
);  
latitudeFeed->save(  
    latitude  
);  
longitudeFeed->save(  
    longitude  
);
```

```
satellitesFeed->save(
    satellites
);
gasAlertFeed->save(
    gasAlert ? 1 : 0
);
Serial.println(
    "Data sent successfully!"
);
// =====
// OLED
// =====
display.clearDisplay();
display.setTextColor(
    SSD1306_WHITE
);
display.setTextSize(1);
display.setCursor(0, 0)
display.print("T:");
display.print(
    temperature,
    1
);
display.print("C ")
display.print("H:")
display.print(
    humidity,
    0
);
```

```
display.println("%")
display.setCursor(0, 15)
display.print("Pulse: ")
display.println(
    pulseValue
);
display.setCursor(0, 30)
display.print("MQ2: ")
display.println(
    mq2Value
);
display.setCursor(0, 45)
if (gasAlert)
{
    display.println(
        "GAS ALERT!"
    );
}
else
{
    display.println(
        "Gas Normal"
    );
}
display.display();
Serial.println(
    "=====
);
```

9. System Working :

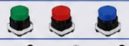
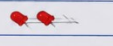


When the system is powered ON, the ESP8266 initializes all the connected sensors and output devices. The controller then establishes a Wi-Fi connection.

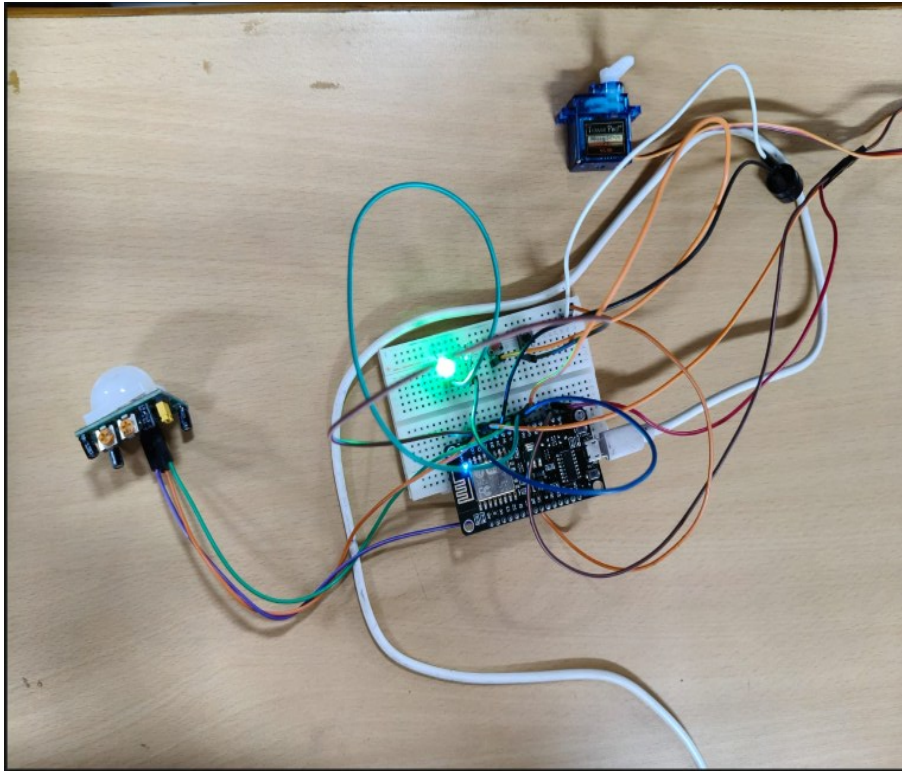
The temperature sensor measures the ambient temperature and humidity. The gas sensor detects potentially harmful gases. A motion or fall sensor can detect abnormal movement or a possible fall.

If an unsafe condition is detected, the red LED turns ON and the buzzer generates an audible warning. At the same time, the sensor information can be transmitted to a cloud platform for remote monitoring.

10. BILL OF MATERIAL:

BILL OF MATERIAL				
S. No.	COMPONENT	SPECIFICATION	QUANTITY	DESCRIPTION / IMAGE
1.	ESP32 Dev Board	ESP32-WROOM-32	1	
2.	PIR Sensor (HC-SR501)	5V DC, 120° Detection	1	
3.	Floor Buttons	Normally Open (NO)	3	
4.	IR Floor Sensors	3.3 - 5V DC	3	
5.	L298N Motor Driver	Dual H-Bridge Driver	1	
6.	DC Geared Motor	12V DC, 60 RPM	1	
7.	16x2 LCD Display (I2C)	16x2 Characters, I2C	1	
8.	Servo Motor (SG90)	9g Micro Servo	1	
9.	Buzzer	5V Active Buzzer	1	
10.	LED (Green)	3mm LED	2	
11.	LED (Red)	3mm LED	2	
12.	Resistors	220Ω, 1/4W	6	
13.	Jumper Wires	Male to Male	As required	
14.	Power Supply	5V DC (2A Adapter)	1	

11. Prototype Implementation:



The prototype was developed using an ESP32 microcontroller, PIR sensor, DC geared motor, motor driver, floor sensors, push buttons, LCD, servo motor, buzzer, and LEDs. A small elevator cabin was constructed with a pulley mechanism connected to the motor for upward and downward movement.

The PIR sensor detects human movement and sends the signal to the ESP32. Based on the selected floor, the ESP32 controls the motor through the motor driver. Floor sensors detect the elevator position and stop the motor at the required floor.

The complete prototype was assembled and tested for human detection, floor selection, elevator movement, floor positioning, and automatic door operation. The system successfully demonstrated the basic working principle of a smart automated elevator.

12. Results and Performance Analysis:

The developed Smart Elevator Automation and Human Detection System using PIR Sensor was successfully implemented and tested to evaluate its functionality, reliability, response, and safety-related operations. The prototype integrates a PIR sensor for human movement detection, an ESP32 microcontroller for decision-making and control, floor-selection switches for user input, floor sensors for position feedback, an L298N motor driver for motor control, a DC geared motor for elevator movement, and a servo motor for automatic door operation

During the initial testing phase, each component was tested individually before integrating the complete system. The PIR sensor was tested by introducing a person into the detection area and observing the output signal generated by the sensor. The sensor successfully detected human movement and transmitted the corresponding digital signal to the ESP32. The response of the sensor was dependent on the movement of the person, sensor sensitivity, detection range, and mounting position.

The prototype also provides opportunities for improving energy efficiency. The controller can place the elevator system in a standby condition when no user activity is detected. Unnecessary motor operation can therefore be avoided. Similarly, the controller can be programmed to activate specific components only when required. These strategies can reduce unnecessary power consumption and improve the efficiency of the system.

The performance of the system is influenced by several mechanical and electrical factors. Motor speed affects the time required for the elevator to travel between floors, while the weight of the cabin affects the torque required from the motor. Friction in the guide mechanism, pulley alignment, cable tension, power-supply stability, and sensor positioning can also influence the overall performance.

the experimental evaluation demonstrates that the proposed Smart Elevator Automation and Human Detection System using PIR Sensor is capable of performing the basic functions required for an automated elevator prototype.

The system successfully demonstrates human-motion detection, floor selection, bidirectional movement, floor-position detection, automatic stopping, and door control.

Performance Analysis

The prototype performed successfully in the following areas:

- **Human Detection:** The PIR sensor successfully detected human movement within the elevator cabin.
- **Automatic Floor Selection:** The system correctly identified the floor selected by the user through the push buttons.
- **Elevator Movement:** The DC geared motor successfully moved the elevator cabin in both upward and downward directions.
- **Floor Detection:** The floor sensors accurately detected the position of the elevator cabin at the respective floors.
- **Automatic Motor Control:** The ESP32 controlled the motor through the L298N motor driver and stopped the elevator when the required floor was reached.
- **Automatic Door Operation:** The servo motor successfully opened and closed the elevator door at the destination floor.
- **Real-Time Control:** The ESP32 processed sensor inputs and generated control outputs with minimal delay.
- **System Integration:** All major components worked together successfully as a single automated elevator system.
- **Repeatability:** The prototype was able to perform multiple floor-selection and movement cycles with consistent operation.

The overall reliability of the system was evaluated by repeatedly performing elevator operations between different floors. Both upward and downward movements were tested. The system was able to coordinate the PIR sensor, floor-selection buttons, floor sensors, motor driver, motor, and door mechanism according to the programmed sequence. This indicates that the integration of hardware and software was successful.